

Chapter 3: Edge Detection

Assoc. Prof. Dr.-Ing. Nguyễn Xuân Hà

Robotics Group, Dept. of Mechatronics, SME - HUST

ha.nguyenxuan@hust.edu.vn

<https://sites.google.com/site/tsnguyenxuanhabkhn/home>

Lecture 3: Programming exercise

Bài 1: Cho ảnh như hình dưới đây (ảnh 'C3_B1.png' trong thư mục data đi kèm).



Thực hiện tính toán đạo hàm của ảnh theo phương x và phương y bằng cách sử dụng phép tích chập với hai ma trận lọc sau:

$$K_x = \frac{1}{3} \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix},$$

$$K_y = \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Thực hiện tính toán độ lớn và hướng của gradient.

Lecture 3: Programming exercise

Chương trình

```
img_dir = './data/C3_B1.png'
```

```
img = cv2.imread(img_dir) -----> Đọc ảnh
```

```
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) -----> Chuyển sang  
ảnh xám
```

```
img_gray = img_gray.astype(np.float64) -----> Chuyển giá trị  
sang số thực
```

Lecture 3: Programming exercise

Chương trình

```
Kx = np.array([  
    [1, 0, -1],  
    [1, 0, -1],  
    [1, 0, -1]  
])
```

```
Kx = Kx / 3
```

```
Ky = np.array([  
    [1, 1, 1],  
    [0, 0, 0],  
    [-1, -1, -1]  
])
```

```
Ky = Ky / 3
```

Ma trận lọc K_x

Ma trận lọc K_y

Lecture 3: Programming exercise

Chương trình

`Ix = cv2.filter2D(img_gray, -1, Kx)` -----> Tính toán đạo hàm ảnh theo phương ngang

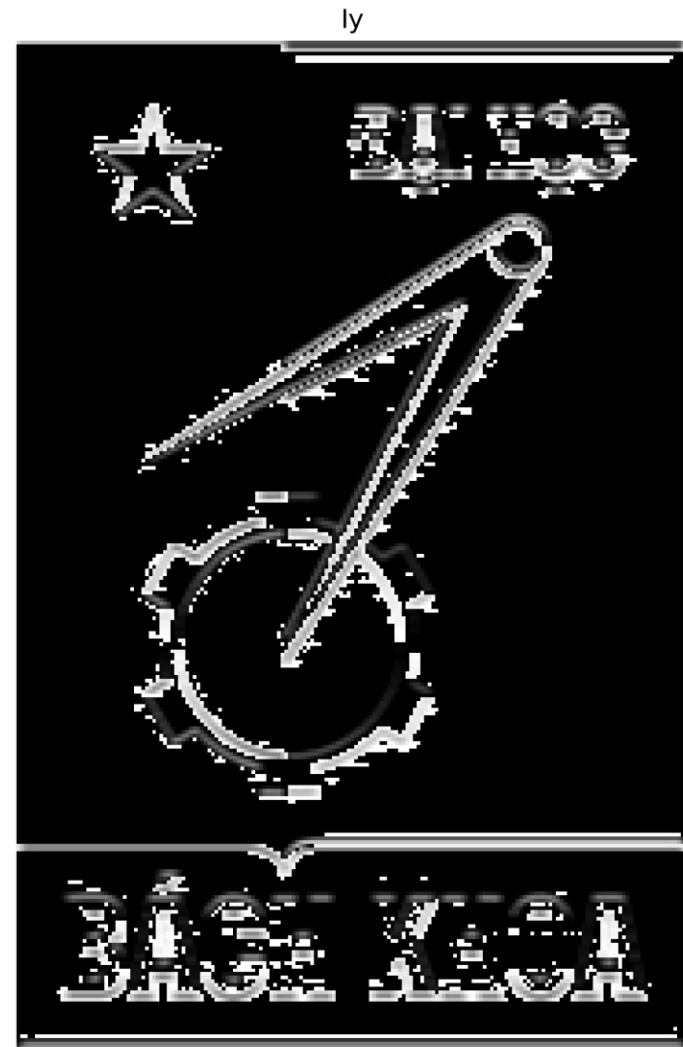
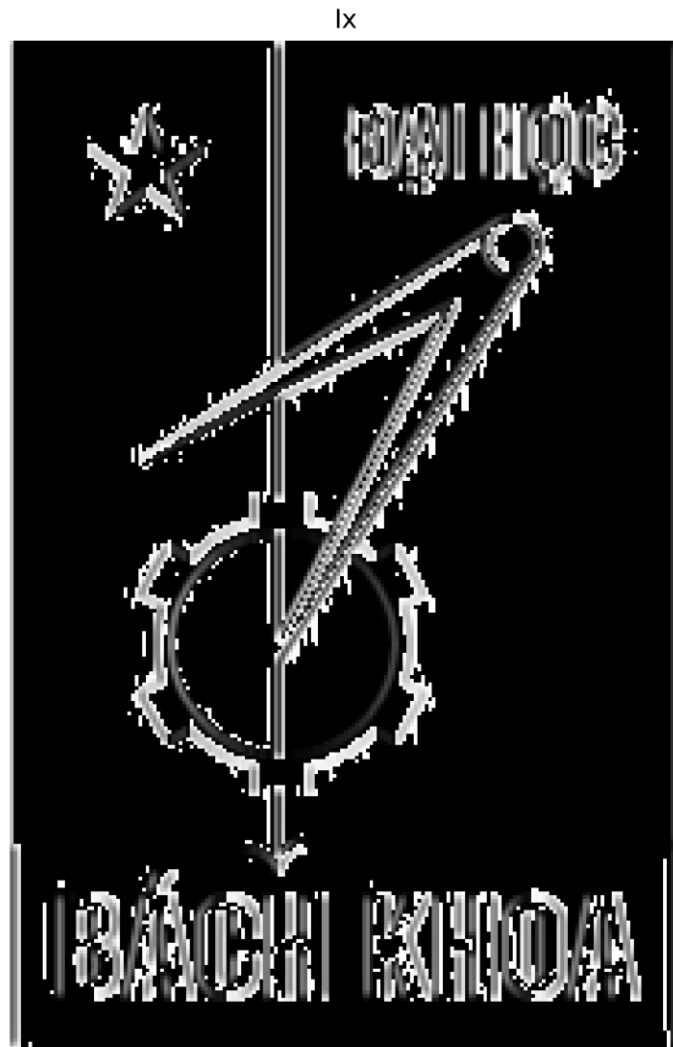
`Iy = cv2.filter2D(img_gray, -1, Ky)` -----> Tính toán đạo hàm ảnh theo phương dọc

`A = (Ix**2 + Iy**2)**0.5` -----> Tính toán độ lớn gradient

`Theta = np.arctan2(Iy, Ix)`

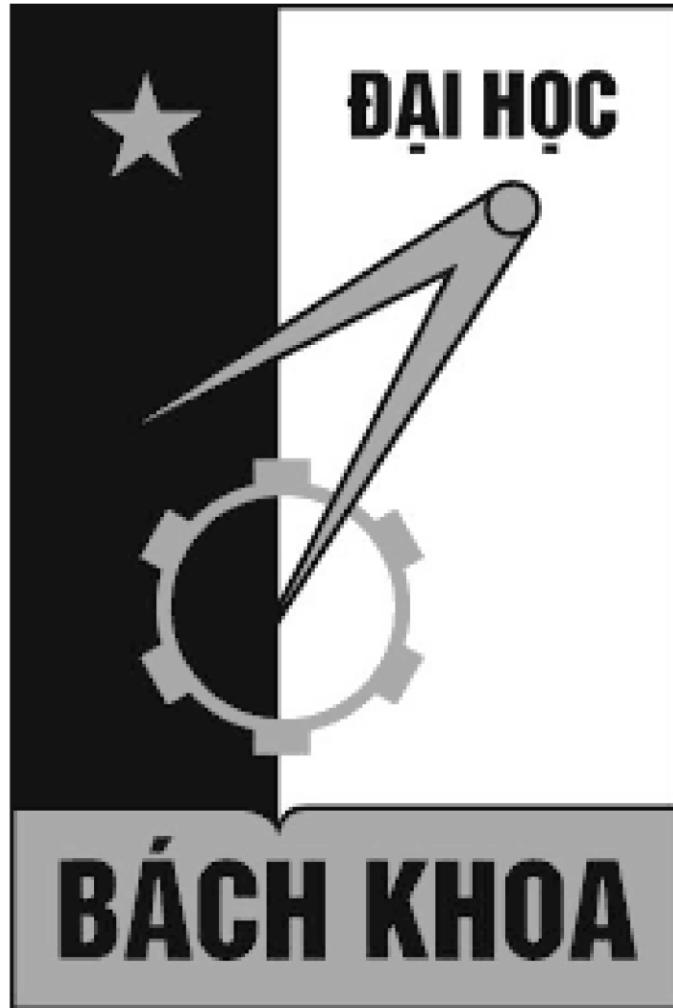
`Theta_degree = Theta * 180 / np.pi` } -----> Tính toán hướng của gradient

Lecture 3: Programming exercise

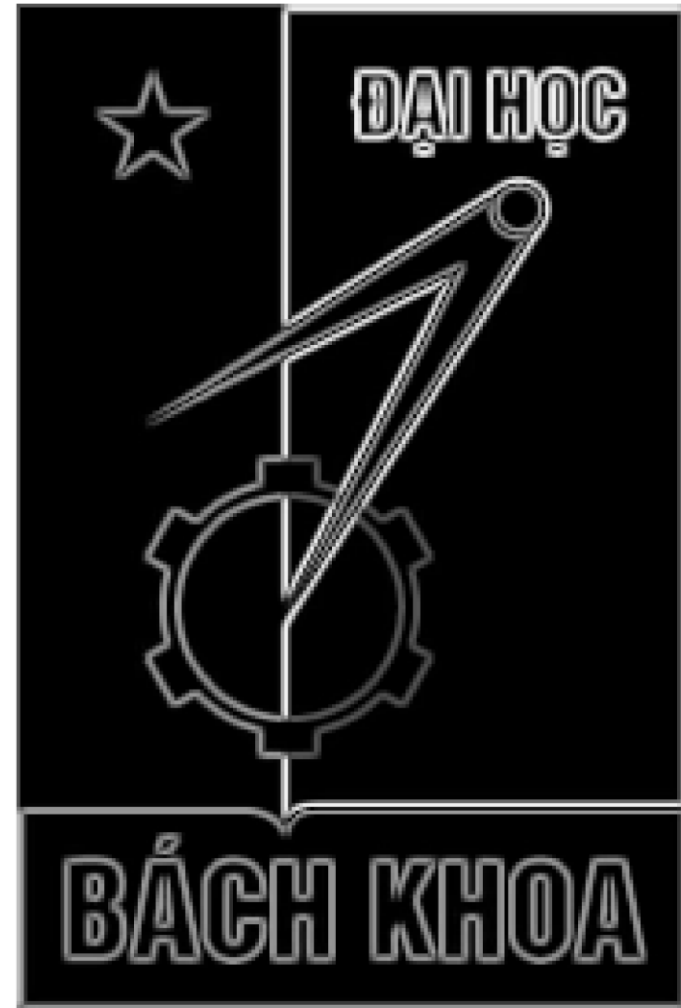


Lecture 3: Programming exercise

Anh xam



A



Lecture 3: Programming exercise

Bài 2: Cho ảnh như hình dưới đây (ảnh 'C3_B2.png' trong thư mục data đi kèm).



Thực hiện tính toán đạo hàm của ảnh theo phương x và phương y bằng cách sử dụng toán tử Sobel:

$$S_x = \frac{1}{8} \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

$$S_y = \frac{1}{8} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Thực hiện tính toán độ lớn và hướng của gradient.

Lecture 3: Programming exercise

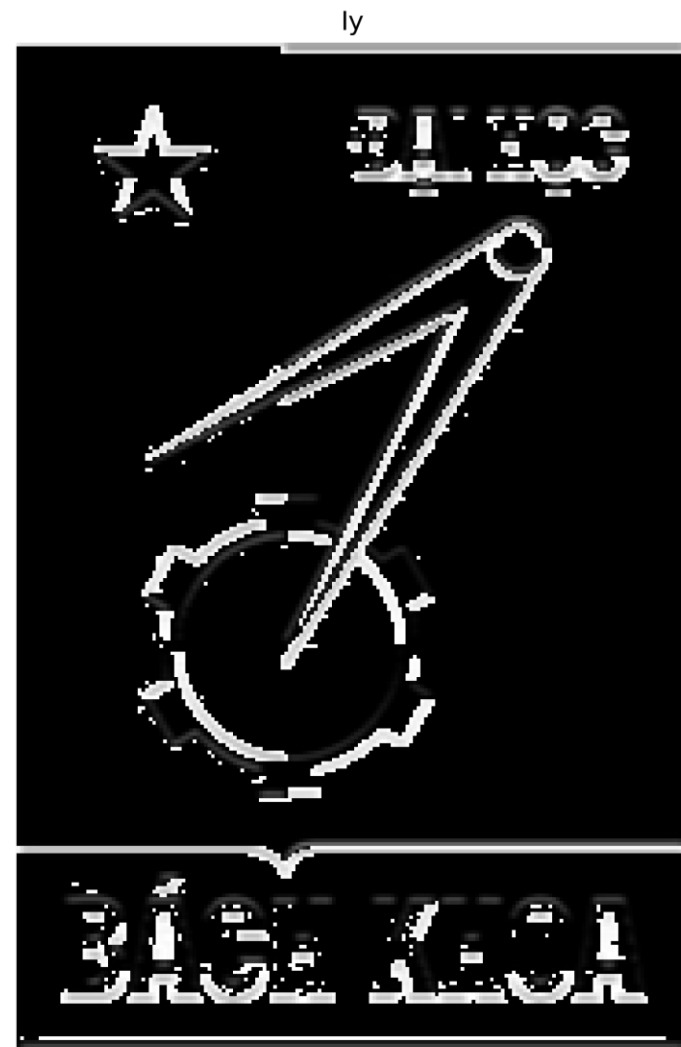
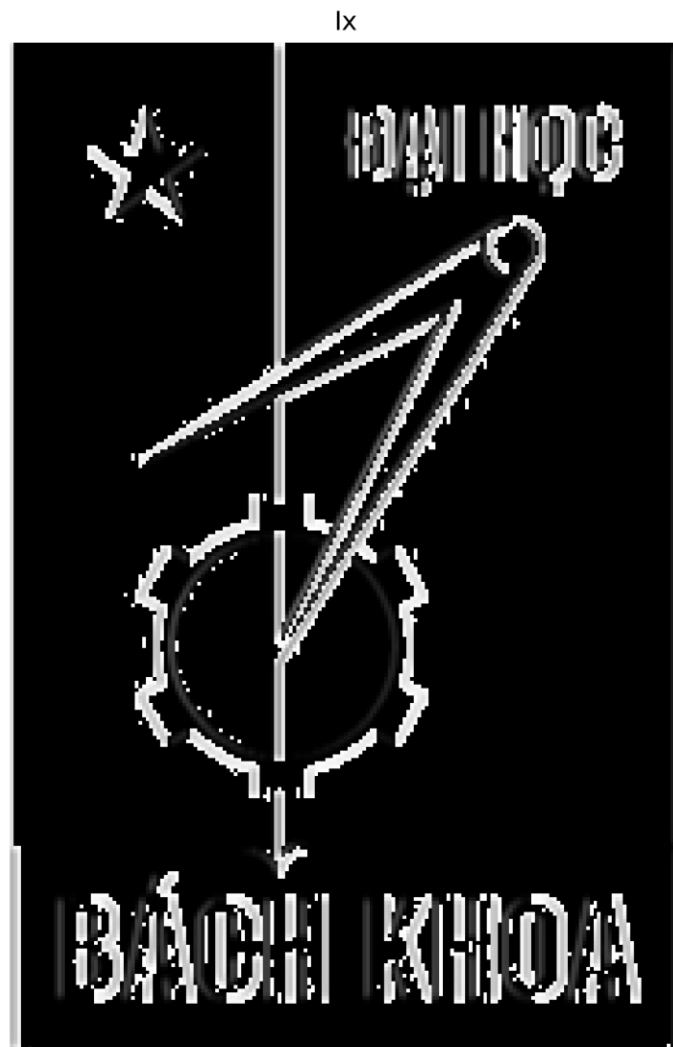
Chương trình

```
Sx = np.array([  
    [1, 0, -1],  
    [2, 0, -2],  
    [1, 0, -1]  
])  
Sx = Sx / 8  
Sy = np.array([  
    [1, 2, 1],  
    [0, 0, 0],  
    [-1, -2, -1]  
])  
Sy = Sy / 8
```

Ma trận lọc S_x

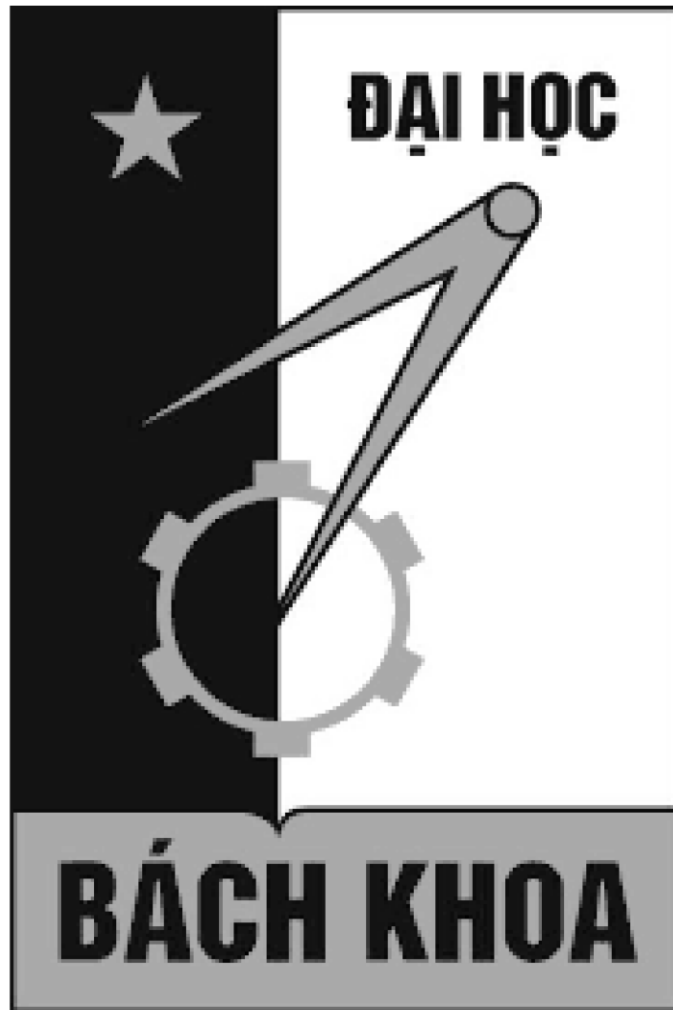
Ma trận lọc S_y

Lecture 3: Programming exercise

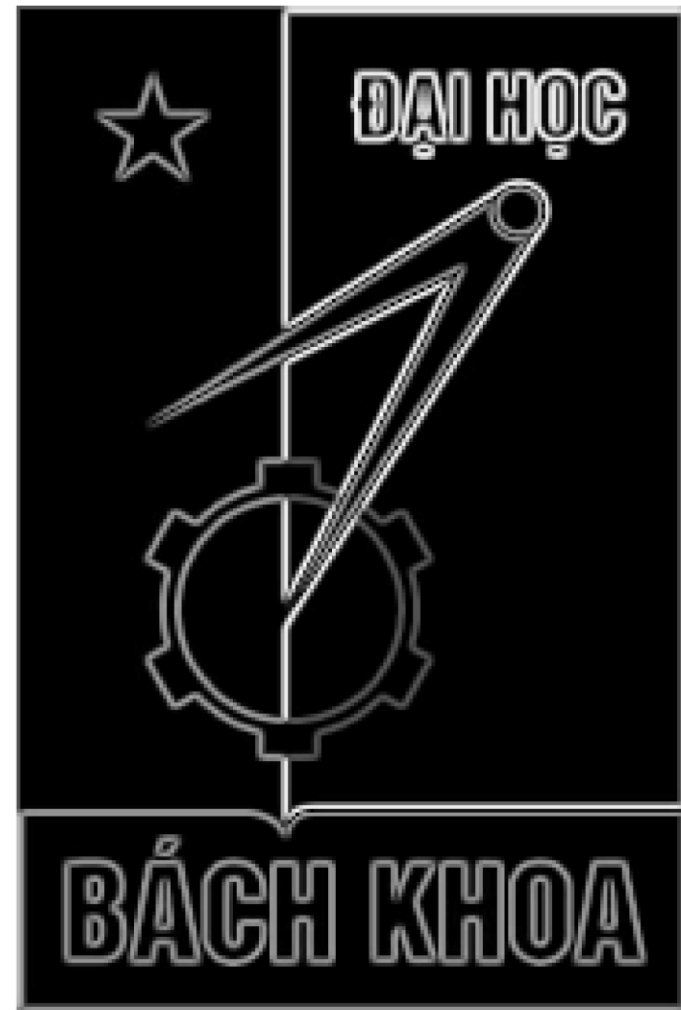


Lecture 3: Programming exercise

Anh xam



A



Lecture 3: Programming exercise

Bài 3: Cho ảnh như hình dưới đây (ảnh 'C3_B3.png' trong thư mục data đi kèm).



Viết chương trình áp dụng thuật toán phát hiện cạnh Canny theo các bước:

- Bước 1: Loại bỏ nhiễu;
- Bước 2: Tính toán độ lớn, hướng gradient;
- Bước 3: Non-Maximum Suppression;
- Bước 4: Lọc ngưỡng;
- Bước 5: Theo dõi cạnh theo độ trễ.

So sánh kết quả thu được với câu lệnh có sẵn trong thư viện opencv

Lecture 3: Programming exercise

```
def GetGradientMatrix(img):  
    img = img.astype(np.float64)
```

-----> Chuyển từ giá trị nguyên sang thực

```
    Sx = np.array([  
        [1, 0, -1],  
        [2, 0, -2],  
        [1, 0, -1]  
    ])
```

```
    Sy = np.array([  
        [1, 2, 1],  
        [0, 0, 0],  
        [-1, -2, -1]  
    ])
```

-----> Khai báo các ma trận lọc Sobel

```
    Ix = cv2.filter2D(img, -1, Sx)  
    Iy = cv2.filter2D(img, -1, Sy)
```

-----> Tính toán kết quả sau khi qua bộ lọc Sobel

```
    A = (Ix**2 + Iy**2)**(1/2)  
    Theta = np.arctan2(Iy, Ix)  
    Theta = Theta * 180 / np.pi
```

-----> Tính toán độ lớn và hướng của gradient

```
    return A, Theta
```

Lecture 3: Programming exercise

```
def NonMaximumSuppression(A, Theta):  
    Theta[Theta < 0] += 180  
    Theta[Theta <= 22.5] = 0  
    Theta[(Theta > 22.5) * (Theta <= 67.5)] = 45  
    Theta[(Theta > 67.5) * (Theta <= 112.5)] = 90  
    Theta[(Theta > 112.5) * (Theta <= 157.5)] = 135  
    Theta[Theta > 157.5] = 0
```

Đưa các giá trị
hướng về các góc
0, 45°, 90°, 135°

```
Anew = np.zeros_like(A)  
for i in range(1, A.shape[0]-1):  
    for j in range(1, A.shape[1]-1):  
        if Theta[i, j] == 0:  
            if A[i, j] >= A[i, j+1] and A[i, j] >= A[i, j-1]:  
                Anew[i, j] = A[i, j]  
        elif Theta[i, j] == 45:  
            if A[i, j] >= A[i+1, j+1] and A[i, j] >= A[i-1, j-1]:  
                Anew[i, j] = A[i, j]  
        elif Theta[i, j] == 90:  
            if A[i, j] >= A[i+1, j] and A[i, j] >= A[i-1, j]:  
                Anew[i, j] = A[i, j]  
        elif Theta[i, j] == 135:  
            if A[i, j] >= A[i-1, j+1] and A[i, j] >= A[i+1, j-1]:  
                Anew[i, j] = A[i, j]
```

Xử lý khi góc
hướng là 0°

Xử lý khi góc
hướng là 45°

Xử lý khi góc
hướng là 90°

Xử lý khi góc
hướng là 135°

```
return Anew
```

Lecture 3: Programming exercise

```
def TracingEdges(A, lower, higher):
```

```
    A1 = np.zeros_like(A)
```

```
    A1[A <= lower] = 0
```

```
    A1[(lower < A) * (A < higher)] = 1
```

```
    A1[A >= higher] = 2
```

```
    A2 = A1.copy()
```

Áp dụng ngưỡng kép cho các giá trị

```
    for i in range(1, A1.shape[0]-1):
```

```
        for j in range(1, A1.shape[1]-1):
```

```
            if A1[i, j] == 1:
```

```
                temp = A1[i-1:i+2, j-1:j+2]
```

```
                if np.max(temp) == 2:
```

```
                    A2[i, j] = 2
```

```
            else:
```

```
                A2[i, j] = 0
```

Xem xét các giá trị pixel ngoại lai

```
    A3 = A2.copy()
```

```
    A3[A2 == 2] = 255
```

```
    A3 = A3.astype(np.uint8)
```

```
    return A3
```

Chuyển thành ảnh đen trắng

Lecture 3: Programming exercise

Ảnh xám Giá trị σ cho ma trận lọc Gaussian Hai giá trị ngưỡng kép

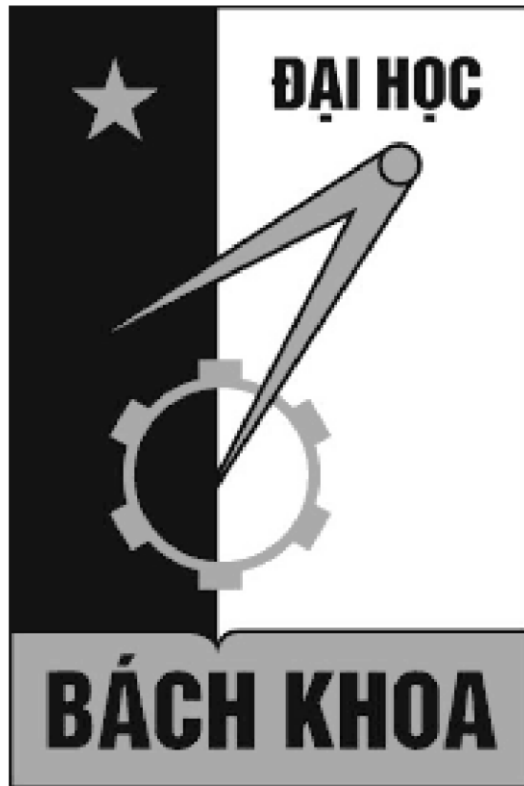
```
def CannyEdgeDetection(img1, sigma, LowerThreshold, HigherThreshold):  
    img_blurr = cv2.GaussianBlur(img1, (5, 5), sigma)  -----> Lọc nhiễu  
    A, Theta = GetGradientMatrix(img_blurr) -----> Tính độ lớn, hướng gradient  
    A_new = NonMaximumSuppression(A, Theta) -----> Non-Maximum Suppression  
    img2 = TracingEdges(A_new, LowerThreshold, HigherThreshold) -----> Lọc ngưỡng + Theo dõi cạnh theo độ trễ  
    return img2
```

```
img_blur = cv2.GaussianBlur(img_gray, (5, 5), 0.5) -----> Lọc nhiễu
```

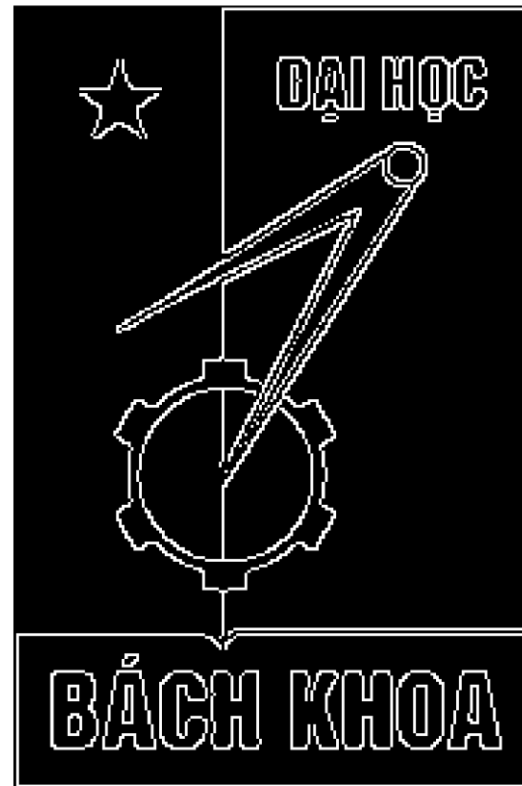
```
edge = cv2.Canny(img_blur, 20, 50) -----> Phát hiện cạnh bằng thuật toán Canny trong opencv
```

Lecture 3: Programming exercise

Anh xam



Cong thuc



opencv

